Bootstrap Identities, Genesis Emissions, and Blind Entropy Safety

Launch-Phase Architecture for DSM

March 12, 2026

1 Bootstrap Identities, Genesis Emissions, and Blind Entropy Safety

This document specifies the launch-phase architecture for DSM when normal live identity creation depends on storage-node-assisted blind entropy and hardware-bound commitment logic. It addresses four distinct matters:

(i) the fixed native-token supply,
(ii) the genesis emission and allocation model,
(iii) the temporary bootstrap regime required to instantiate the first storage nodes, and
(iv) the security boundary that prevents launch exceptions from becoming permanent trust assumptions.

The launch problem is circular in only one narrow sense: ordinary live identities depend on storage-node participation, while the first storage nodes must already exist in order to provide that participation. The solution is a strictly transitional bootstrap regime. A small initial set of bootstrap operators exists only to instantiate the minimum storage fabric required for the ordinary live regime. Once the network can satisfy liveness and registration requirements without them, those bootstrap operators enter deterministic drain and unregistry.

This infrastructure problem should not be confused with the token-distribution problem. The native token itself can be fully predetermined at genesis. Its supply surface, emission rules, and initial distribution commitments can all be fixed in advance and shipped as part of the canonical protocol surface. The distribution side of this model should be read together with the DJTE emissions paper [1].

1.1 Fixed Native Token Supply

The native token is defined by a canonical immutable CPTA with a fixed total supply of

$$80{,}000{,}000{,}000$$

units.

This total supply is predetermined at genesis and is not altered later by governance, operator discretion, or mutable monetary policy. The token surface is frozen by the canonical policy commitment exactly as any other immutable CPTA object is frozen.

Accordingly, the native token is specified by:

- a fixed token identity,
- a fixed policy commitment,
- a fixed total supply,
- a fixed emission framework, and

- fixed genesis allocation rules.

The token therefore does not require a social mint event or discretionary launch issuance. It can exist as a predetermined canonical object from day one.

1.2 Genesis Emissions

The initial token surface is introduced through genesis emissions. These emissions are not ad hoc mint events. They are predetermined allocations or reveals under the native token's canonical policy.

At the distribution layer, this composes naturally with DJTE: emission eligibility is unlocked by deterministic activation proofs, activation consumption is single-use, supply growth is bounded by policy, and the system remains proof-carrying even when the activated identity set is still small during the bootstrap era [1].

The purpose of the initial emissions is straightforward:

(i) place balances into circulation at genesis,
(ii) allow early participants to stake or acquire network resources,
(iii) seed liquidity and operational activity, and
(iv) avoid dependence on a tiny manually privileged operator set.

Some recipients will stake to run storage nodes. Some will provide liquidity. Some will hold or transfer their balances. These are consequences of distribution, not protocol privileges. Token ownership alone does not create additional protocol authority merely because it began at genesis.

1.3 Genesis Allocation Form

The genesis allocation may be represented in one of two equivalent ways.

1.3.1 Option A: Direct Genesis Recipient Set

Genesis directly assigns balances to a fixed set of precommitted recipient endpoints.

1.3.2 Option B: Genesis Claim Commitments

Genesis assigns balances to claim commitments or bootstrap claim containers that are later claimed by recipients once the first minimal storage fabric is live.

The second model is cleaner for DSM because it avoids forcing fully live post-launch identity generation to exist before the first storage nodes exist.

Under this model:

- genesis defines a predetermined set of claim commitments,
- each commitment corresponds to a fixed allocation,
- those commitments are claimable by intended recipients, and
- once the minimum live storage set exists, recipients bind the claim into a normal live identity and migrate the funds.

This breaks the circular dependency cleanly.

1.4 Bootstrap Claim Endpoints

If normal live identity creation depends on storage-node-assisted blind entropy, then the very first recipient endpoints cannot all be created through the ordinary live process. That would be circular.

To solve this, genesis may use *bootstrap claim endpoints* or *bootstrap claim containers*. These are transitional objects whose sole purpose is to receive genesis allocation before the ordinary live

identity path is available. They are not intended to function forever as full canonical long-term identities.

A bootstrap claim endpoint may therefore:

- receive a genesis allocation,
- prove ownership of that allocation, and
- migrate that allocation into a normal live identity once available.

Its role is transitional only.

Remark 1.1 (Migration-Only Bootstrap Endpoints). Bootstrap claim endpoints may exist in genesis without storage-node-assisted entropy, but they are transitional only. Their sole normative purpose is to receive genesis allocation and deterministically migrate that allocation into a storage-node-derived live identity, after which the bootstrap endpoint is exhausted and must not be used as a continuing canonical holder.

This removes the chicken-and-egg problem without polluting the steady-state model.

1.5 Migration of Bootstrap Balances

Once the minimal storage fabric exists and normal live identities can be created, bootstrap allocations are expected to be fully bled into proper live identities.

This means:

- the bootstrap claim endpoint receives genesis value,
- the holder later creates a proper live identity using the normal entropy path,
- the full balance is migrated into that live identity, and
- the bootstrap claim endpoint is exhausted and retired.

This preserves the integrity of the runtime trust model. The protocol is not saying that non-live bootstrap endpoints are equivalent forever to ordinary long-term identities. It is saying that they are temporary landing zones used only to bridge genesis into the normal live system.

1.6 Bootstrap Storage Node Operators

The first storage nodes must exist before ordinary live identity creation becomes available. Therefore, a small set of bootstrap operator identities may be introduced for the sole purpose of bringing up the initial storage fabric.

These bootstrap operator identities are not permanent protocol aristocracy. They are temporary registry placeholders. Their purpose is narrow:

- stand up the minimum initial storage set,
- participate in the initial blind entropy path,
- allow live identity creation to become possible, and
- then prepare for replacement by normal operators.

They should therefore be treated as:

- transitional infrastructure identities,
- not privileged founding accounts,
- not permanent meta-authorities, and
- not inherently canonical long-term participants.

A useful conceptual tag is:

`BOOTSTRAP_OPERATOR`

1.7 Relationship Between Genesis Allocation and Bootstrap Operators

A bootstrap regime does not require permanent staking exemptions if genesis distribution is designed correctly.

Because the native token supply is predetermined and the genesis emissions are pre-programmed, recipients can begin the network already holding balances. Those balances can be used for:

- staking,
- resource acquisition,
- early storage registration,
- liquidity formation, and
- ordinary holding.

This means there are three coherent launch models.

1.7.1 Model 1: No Exemption, Genesis Airdrop Funds Staking

This is the cleanest model:

- the canonical native token exists at genesis,
- the 80 billion supply is predetermined,
- genesis allocations are predetermined,
- recipients use those balances to stake immediately, and
- early storage nodes enter under the same stake rule as everyone else.

This is architecturally strongest because the staking rule is universal from day one.

1.7.2 Model 2: Temporary Bootstrap Exemption

This is operationally simpler but less elegant:

- bootstrap operators are temporarily allowed to run without stake,
- ordinary staking rules activate once the network is large enough, and
- bootstrap identities are later retired or re-enter normally.

This works, but it introduces asymmetry into the launch era.

1.7.3 Model 3: Reserved Infrastructure Allocation

This is a middle path:

- a fixed tranche of the genesis emissions is reserved for launch infrastructure,
- those tokens satisfy the initial stake requirements for bootstrap operators,
- the staking rule itself never changes, and
- the initial allocation simply accounts for infrastructure bring-up.

Of the three, the cleanest long-term architecture is either Model 1 or Model 3, because the protocol rule itself stays universal.

1.8 Bootstrap Operator Lifecycle

Bootstrap operators should have a deterministic lifecycle rather than an indefinite special status. A clean lifecycle is:

$$\texttt{BootstrapRegistered} \rightarrow \texttt{ReplacementEligible} \rightarrow \texttt{PendingDrain} \rightarrow \texttt{Unregistered}$$

BootstrapRegistered The operator is active only because the system still requires transitional infrastructure.

ReplacementEligible The network can now satisfy liveness and registration requirements without that bootstrap operator.

PendingDrain The operator stops receiving new long-horizon responsibilities. Data, assignments, and availability obligations are re-mirrored away.

Unregistered The bootstrap operator exits the bootstrap role entirely.

This must be deterministic, not discretionary. The system should not depend on a social decision about when the bootstrap era is "over."

1.9 Deterministic Replacement Conditions

A bootstrap operator should become replaceable only when the network can prove that removing that operator still leaves the system above all required thresholds.

Typical conditions include:

- the minimum live node count has been exceeded by a safety margin,
- replica and availability requirements are satisfied without relying on that operator,
- enough non-bootstrap identities exist to fill the relevant roles, and
- no irreplaceable bootstrap-only service remains.

Replacement should be per-node, not all-at-once. This is cleaner because it:

- avoids cliff transitions,
- preserves liveness more smoothly,
- reduces coordination risk, and
- eliminates ceremonial "end of bootstrap era" switches.

Remark 1.2 (Disposable Bootstrap Operators). Bootstrap operator identities exist solely to instantiate the minimum initial storage fabric required for live identity creation and normal network operation. They are transitional infrastructure identities, not privileged permanent participants. Once the network can satisfy all registration, availability, and replacement thresholds without a given bootstrap node, that node enters replacement eligibility, ceases receiving new long-horizon assignments, and prepares for deterministic drain and unregistry. Any continued participation by the same human operator must occur through a normal live identity under the ordinary registration rules.

1.10 Re-Entry Under Normal Rules

If the human operator behind a bootstrap node wants to continue participating after the bootstrap role is retired, that participation should occur through ordinary re-entry:

- create or bind to a normal live identity,
- satisfy the same registration or staking requirements as everyone else, and
- register again under the ordinary steady-state regime.

This ensures that bootstrap status is structurally disposable and never turns into hereditary privilege.

1.11 Distribution, Decentralization, and Security Are Distinct

A key clarification is that what conventional systems often treat as one combined bootstrap problem is, in DSM, actually three separate problems:

(i) **distribution,**
(ii) **decentralization,** and
(iii) **security.**

These are related, but they are not the same thing.

Distribution Distribution is the question of who initially receives the native token supply. In this system, that can be fully predetermined through the canonical native-token CPTA, fixed total supply, and fixed genesis emissions or airdrop commitments. Distribution is therefore a genesis-state question: who starts with balances, in what quantities, and under what claim or migration rules.

Decentralization Decentralization is the separate question of how quickly and how broadly the live operational network becomes populated by ordinary participants under the normal steady-state rules. That concerns node replacement, live identity creation, registration eligibility, liveness thresholds, and the retirement of transitional bootstrap operators. Decentralization is therefore a network-topology and participation question, not a token-definition question.

Security Security is a third and independent question: whether the launch process preserves the system's trust boundaries while the network is still small. In the present architecture, this concerns at least four issues:

(a) whether transitional bootstrap operators can secretly retain permanent special power,
(b) whether genesis claim endpoints create a lasting side door into the identity model,
(c) whether early storage nodes can secretly reconstruct user secrets during blind entropy participation, and
(d) whether a stale or undersized early network can bias Bitcoin-bound bridge execution.

Traditional systems often collapse all three into one apparent problem because early token ownership is also treated as early control, and early infrastructure is also treated as permanent trust. DSM breaks that coupling. The native token can be distributed at genesis without that distribution automatically conferring permanent weighted authority over the system. Likewise, the early existence of transitional bootstrap operators does not determine the long-term decentralization of the network, provided those operators are deterministically replaceable and hold no enduring privileged role.

So the launch question should be stated correctly:

- **Distribution** asks: who gets the initial token supply?
- **Decentralization** asks: how does the network move from temporary bootstrap infrastructure to ordinary broad participation?
- **Security** asks: does the bootstrap path preserve the same trust boundaries the steady-state system is supposed to enforce?

Because these are separate, they can be solved separately.

1.12 Why Early Token Holders Do Not Gain Special Systemic Power

A common concern is that early genesis recipients or bootstrap operators might gain permanent systemic control merely by being first.

That concern weakens substantially if the system is not governance-weighted by token quantity.

If ownership of larger quantities does not create weighted consensus authority, weighted governance authority, or privileged route acceptance, then early distribution does not automatically translate into lasting protocol domination.

The remaining risks are mostly practical or social:

- early visibility,
- early liquidity concentration,
- early market advantage, and
- optics around who received the first allocations.

Those matter, but they are categorically weaker than a system in which token quantity directly amplifies protocol authority.

Thus the architecture remains clean provided that:

- bootstrap status expires automatically,
- normal operators can replace bootstrap operators deterministically,
- large holders do not gain consensus or governance leverage merely by holding more, and
- staking acts only as a Sybil or commitment gate, not a permanent control multiplier.

1.13 Blind Entropy Safety

A second launch concern is whether storage nodes might be secretly sharing or reconstructing user secrets when they participate in blind entropy generation.

The key point is that storage nodes do not hand the user a finished secret. They only contribute blinded entropy fragments into a deterministic construction. The user's device combines those fragments with its own locally hidden material and hardware-bound commitment data. The final derived secret is never fully known to any individual storage node, and no coalition below the protocol threshold can reconstruct it from node-side observations alone.

In conceptual form:

$$K = H(\text{blind node contributions} \| \text{device-local hidden material} \| \text{hardware commitment} \| \text{domain separation})$$

This means the final secret is not "something the nodes gave the user." It is something derived locally on the device from:

(1) thresholded blind external contributions,
(2) locally hidden device material,
(3) hardware-bound commitment data, and
(4) deterministic domain separation.

1.13.1 What the Safety Claim Actually Is

The safe claim is narrow and precise:

(1) no single node knows the final secret,
(2) no coalition below the protocol threshold can reconstruct it,
(3) the device contributes indispensable hidden material,
(4) the hardware commitment binds the derivation to the concrete device context, and
(5) node participation is contributory, not custodial.

That is the right trust boundary.

Storage nodes do not "hold the user's key in pieces" in any meaningful operational sense. At most, each node sees only:

- its own blinded contribution,
- protocol transcript material, and
- no access to the device's private hidden contribution or final locally derived secret.

Thus the nodes are not secretly sharing the user's actual secret because they do not possess the full secret in the first place.

1.13.2 Role of Hardware Commitment

The hardware commitment is critical because it prevents silent substitution and replay.

Without hardware commitment, people naturally worry that nodes could:

- replay entropy contributions across users,
- reuse old transcripts,
- correlate outputs, or
- steer multiple users into partially related outputs.

Binding derivation to hardware commitment means the output is not simply a reusable entropy blob. It is tied to the concrete device context.

So even if a node replays its own contribution, the final result still changes with:

- the device-local hidden material,
- the hardware-bound commitment, and
- the deterministic domain tag.

This sharply narrows the space for secret substitution, replay, or cross-user correlation.

Remark 1.3 (Blind Entropy Trust Boundary). Storage nodes participate only as blind entropy contributors. They do not generate, possess, or escrow the user's final secret. The device derives the final secret locally by combining thresholded blind contributions with device-local hidden material and hardware-bound commitment inputs under domain-separated hashing. Consequently, no individual node, and no coalition below the protocol threshold, can reconstruct the resulting secret from node-side observations alone. The hardware commitment additionally binds the derivation to the concrete device context, preventing silent substitution, replay, or cross-device reuse of entropy transcripts.

1.14 What Genesis Does Not Require

Genesis does not require that every recipient identity and every future operator identity already be created through the live post-bootstrap entropy path. That would be circular.

Instead, genesis requires only:

- a predetermined token surface,
- a predetermined total supply,
- a predetermined genesis distribution,
- a minimal transitional bootstrap operator set, and
- a deterministic migration path into the normal live regime.

That is enough to launch coherently.

1.15 Complete Launch Sequence

A coherent launch sequence is therefore:

(1) Define the canonical native-token CPTA with fixed supply

$$80{,}000{,}000{,}000$$

and fixed genesis emission logic.

(2) Define the genesis distribution as direct recipient allocations or claim commitments.
(3) Instantiate a minimal set of temporary bootstrap operator identities.
(4) Bring the first storage nodes online.
(5) Make live identity generation available through the normal blind-entropy path.
(6) Allow genesis recipients to migrate bootstrap balances into proper live identities.
(7) Allow those live identities to stake, provide liquidity, hold, or otherwise participate.
(8) As enough normal operators appear, mark bootstrap operators as replacement-eligible.
(9) Drain and unregister bootstrap operators deterministically.
(10) Continue under the ordinary steady-state registration regime.

This resolves the circular dependency without polluting the permanent architecture.

1.16 Summary

The launch model is clean if treated as a transitional regime rather than a permanent exception.

- The native token can be fully pre-programmed.
- The fixed supply can be 80 billion from genesis.
- Initial emissions can function as the airdrop that gives early participants balances for staking, liquidity, storage participation, or simple holding.
- Bootstrap claim endpoints can receive genesis value before live identity creation exists.
- Those bootstrap balances can later be fully migrated into proper live identities.
- Temporary bootstrap operators can bring up the first storage nodes.
- Once the system can replace them safely, they are deterministically drained and retired.

- Blind entropy does not mean nodes secretly share user secrets; they contribute only blinded fragments into a device-local derivation that also depends on hidden local material and hardware commitment.
- The emission side of the launch surface composes naturally with DJTE, which supplies deterministic activation-triggered distribution, proof-carrying validation, and explicit handling of the small-activated-set bootstrap regime.

The result is a system that launches from a fixed deterministic genesis state, solves the first-node chicken-and-egg problem, preserves the integrity of the long-term live identity model, and does so without creating a permanent privileged founding class.

References

[1] B. Ramsay, "DJTE: Deterministic Join-Triggered Emissions for the Deterministic State Machine (DSM): A Proof-Carrying, Consensus-Free Emission System with Shard-Equivalent Global Uniform Sampling with Deterministic Credit Bundles, Contact-Gated Inboxes, and Precommitment-Driven DoS Minimality," Dec. 14, 2025.
https://www.deterministicstatemachine.org/DSM-emissions.pdf